

ICT 1306

OBJECT ORIENTED PROGRAMMING

3.STRUCTURES & FUNCTIONS

TC Irugalbandara
KHA Hettige

Structures

- A structure is a collection of simple variables. The variables in a structure can be of different types.
 - Int, float, char...
- The data items in a structure are called the members of the structure.
- Syntax of a structure is very much identical to that of a class.
- A structure is a collection of data, while a class is a collection of both data and functions

Structure Example

```
// books.cpp
// uses books inventory to demonstrate structures
#include <iostream>
using namespace std;
struct book          //declare a structure
{
    int bookID;        //ID number of the book
    int numOfCopies;    //Number of copies available
    float price;       //cost of a book
};
```

```
int main() {  
    book book1;          //define a structure variable What is the difference with C???  
    book1.bookID = 244; //give values to structure members  
    book1.numOfCopies = 40;  
    book1.price = 296.50F;  
    //display structure members  
    cout << "Book : " << book1.bookID;  
    cout << ", Num of Copies: " << book1.numOfCopies;  
    cout << ", Price $" << book1.price << endl;  
    return 0;  
}
```

Defining the Structure

```
struct book  
{  
    int bookID;  
    int numOfCopies;  
    float price;  
};
```

Structure Name

Members of the structure

Semicolon terminates the definition

- Structure can be identified as the specification for a new data type.
- We can define structure variables same as any other variable.
- Once a structure variable has been defined, its members can be accessed using something called the dot operator (**member access operator**).
 - `book1.bookId;`
- We can initialize member variables when the structure variable is defined.

Initializing Structure Members

```
#include <iostream>
using namespace std;
struct book          //declare a structure
{
    int bookID;        //ID number of the book
    int numOfCopies;    //Number of copies available
    float price;       //cost of a book
};
```

```
int main() {  
    book book1="122,34,234.50";  
    cout << "Book : " << book1.bookID;  
    cout << ", Num of Copies: " << book1.numOfCopies;  
    cout << ", Price $" << book1.price << endl;  
    Book book2=book1; // Assignment is possible  
    cout << "Book : " << book2.bookID;  
    cout << ", Num of Copies: " << book2.numOfCopies;  
    cout << ", Price $" << book2.price << endl;  
    return 0;  
}
```

What is happening in this programme? Explain .

```
#include <iostream>
using namespace std;
struct Distance
{
    int feet;
    float inches;
};
```

```
int main() {
    Distance d1, d3;
    Distance d2 = { 11, 6.25 };
    cout << "\nEnter feet: "; cin >> d1.feet;
    cout << "Enter inches: "; cin >> d1.inches;
    d3 = d1 + d2;
    cout << d1.feet << "'-" << d1.inches << "\" + ";
    cout << d2.feet << "'-" << d2.inches << "\" = ";
    cout << d3.feet << "'-" << d3.inches << "\"\n";
    return 0;
}
```

```

#include <iostream>

using namespace std;

struct Distance
{
    int feet;

    float inches;
};

```

Can't do
 $d_3 = d_1 + d_2$;
 there is no routine built into C++ that knows how
 to add variables of type Distance.

```

int main() {
    Distance d1, d3;
    Distance d2 = { 11, 6.25 };
    cout << "\nEnter feet: "; cin >> d1.feet;
    cout << "Enter inches: "; cin >> d1.inches;
    d3.inches = d1.inches + d2.inches;
    d3.feet = 0;
    if(d3.inches >= 12.0)
    {
        d3.inches -= 12.0;
        d3.feet++;
    }
    d3.feet += d1.feet + d2.feet;
    cout << d1.feet << "\'-" << d1.inches << "\" + ";
    cout << d2.feet << "\'-" << d2.inches << "\" = ";
    cout << d3.feet << "\'-" << d3.inches << "\"\n";
    return 0; }

```

Structures within Structures

Q1. Write a programme to store information about a Room . Room has a width and a length. Type of the With and length are Distance . Then calculate the area of the room and display it.

Structures and Classes in C++

- Structures are usually used to hold data only.
- Classes are used to hold both data and functions.
- However, in C++, structures can hold both data and functions. (In C they can hold only data.)
- The syntactical distinction between structures and classes in C++ is minimal, so they can in theory be used almost interchangeably.

Q2.

Create a structure called time. Its three members, all type int, should be called hours, minutes, and seconds.

Write a program that prompts the user to enter a time value in hours, minutes, and seconds.

The program should then store the time in a variable of type time, and finally print out the total number of seconds represented by this time value.

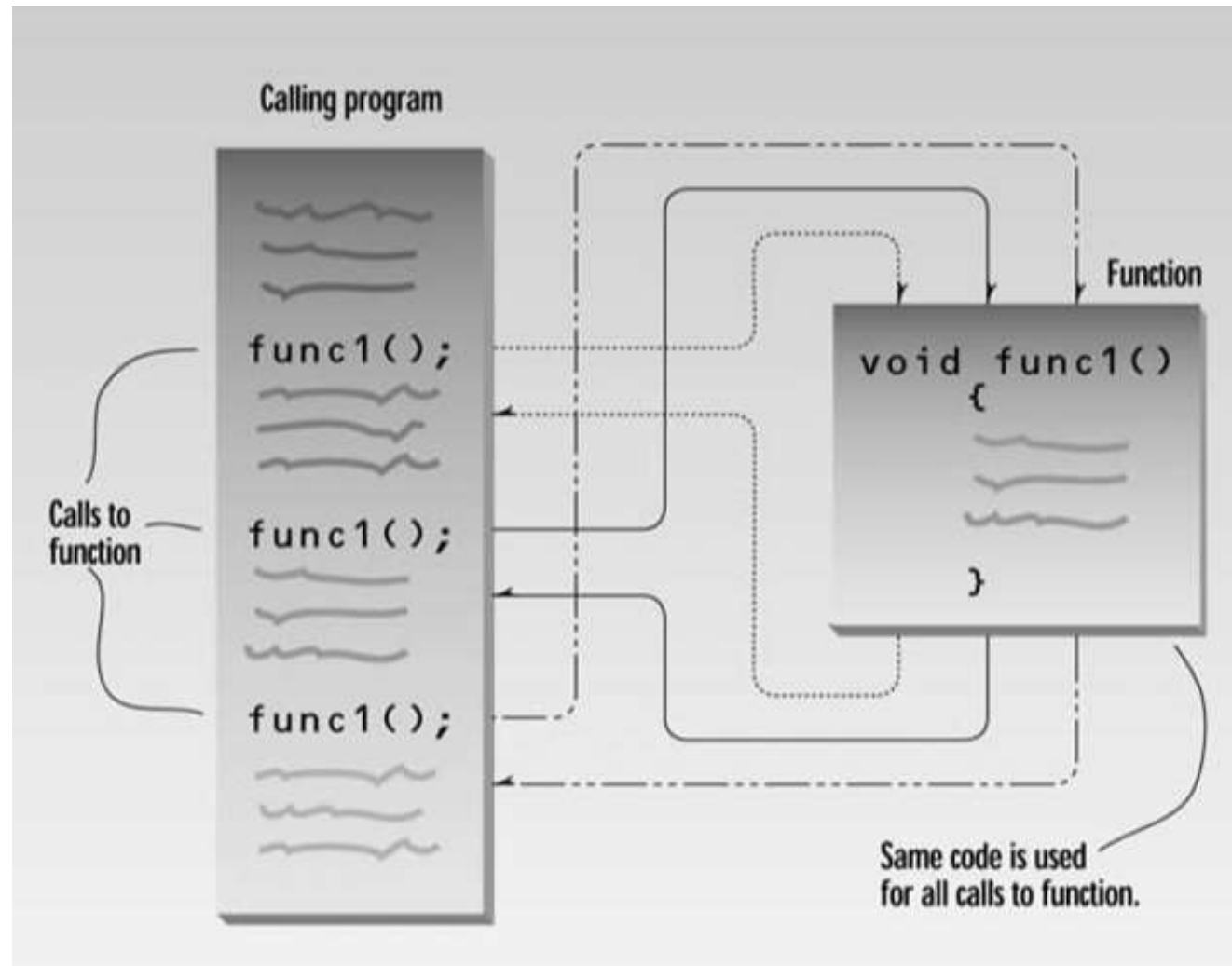
Enumerations

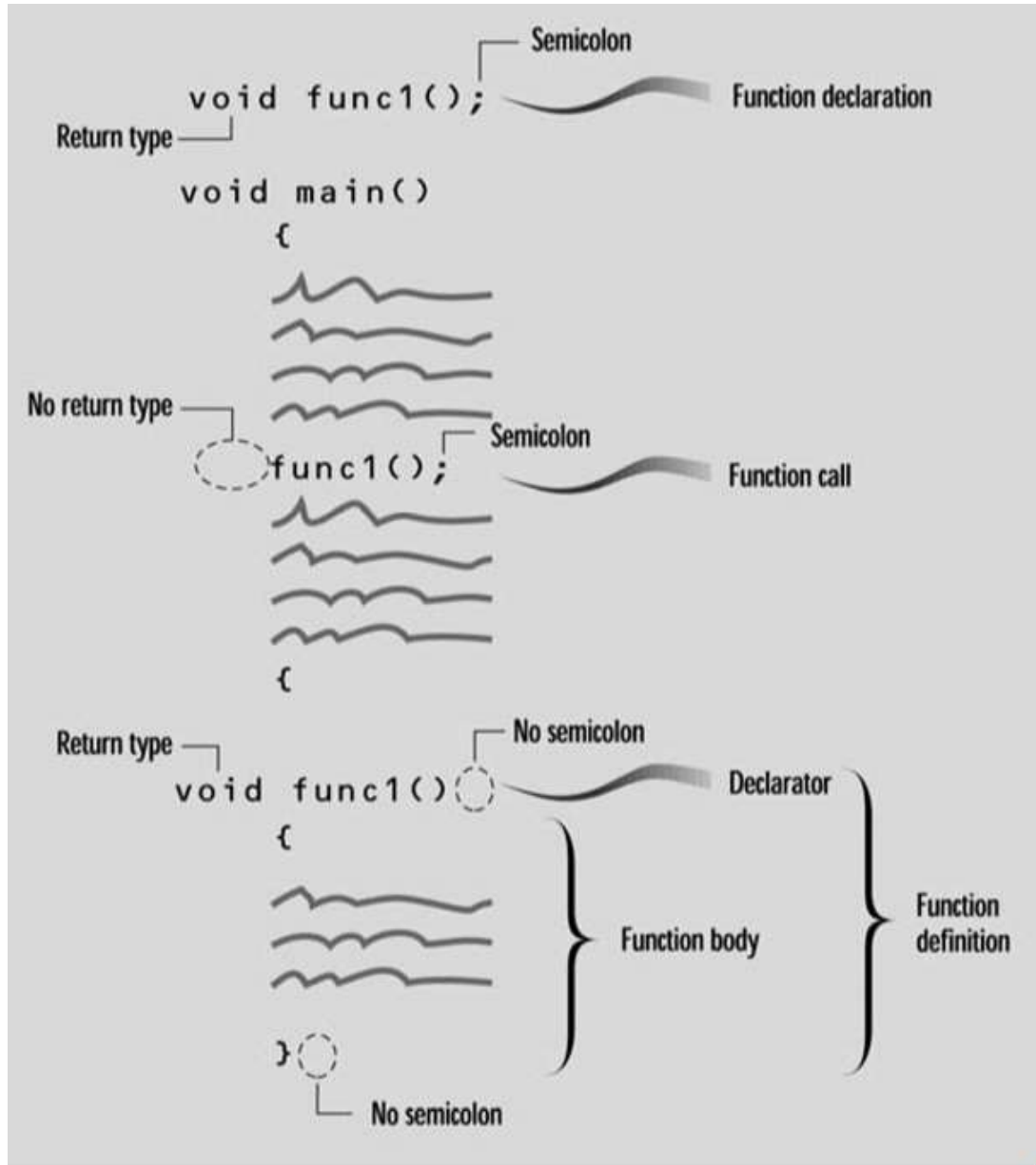
- Enumerated types work when you know in advance a finite (usually short) list of values that a data type can take on.
- An enum declaration defines the set of all names that will be permissible values of the type. These permissible values are called **enumerators**.
- An enumeration is a list of all possible values.
- can't use values that weren't listed in the declaration.
- can use the standard arithmetic operators on enum types.

```
// dayenum.cpp
// demonstrates enum types
#include <iostream>
using namespace std;
enum days_of_week { Sun, Mon, Tue, Wed, Thu, Fri, Sat };
int main() {
    days_of_week day1, day2; //define variables of type days_of_week
    day1 = Mon;           //give values to variables
    day2 = Thu;
    int diff = day2 - day1; //can do integer arithmetic
    cout << "Days between = " << diff << endl;
    if(day1 < day2)        //can do comparisons
        cout << "day1 comes before day2\n";
    return 0;
}
```

Functions

- A function groups a number of program statements into a unit and gives it a name. This unit can then be invoked from other parts of the program.
- Functions use for
 - Conceptual organization of a program
 - Reduce the programme size
- Any sequence of instructions that appears in a program more than once is a candidate for being made into a function.
- The function's code is stored in only one place in memory, even though the function is executed many times in the course of the program





```

// profile.cpp
// demonstrates simple function
#include <iostream>
using namespace std;
void starline();           //function declaration (prototype)
int main() {
    starline();             //call to function
    cout << "Personnel Profile" << endl;
    starline();             //call to function
    cout << "Name: Kamal " << endl
    << "Age: 24 " << endl
    << "DoB : 1992/10/12" << endl;
    starline();             //call to function
    return 0;
}

```

```

//starline() function definition
void starline()             //function declaratory
{
    for(int j=0; j<45; j++)  //function body
        cout << '*';
    cout << endl;
}

```

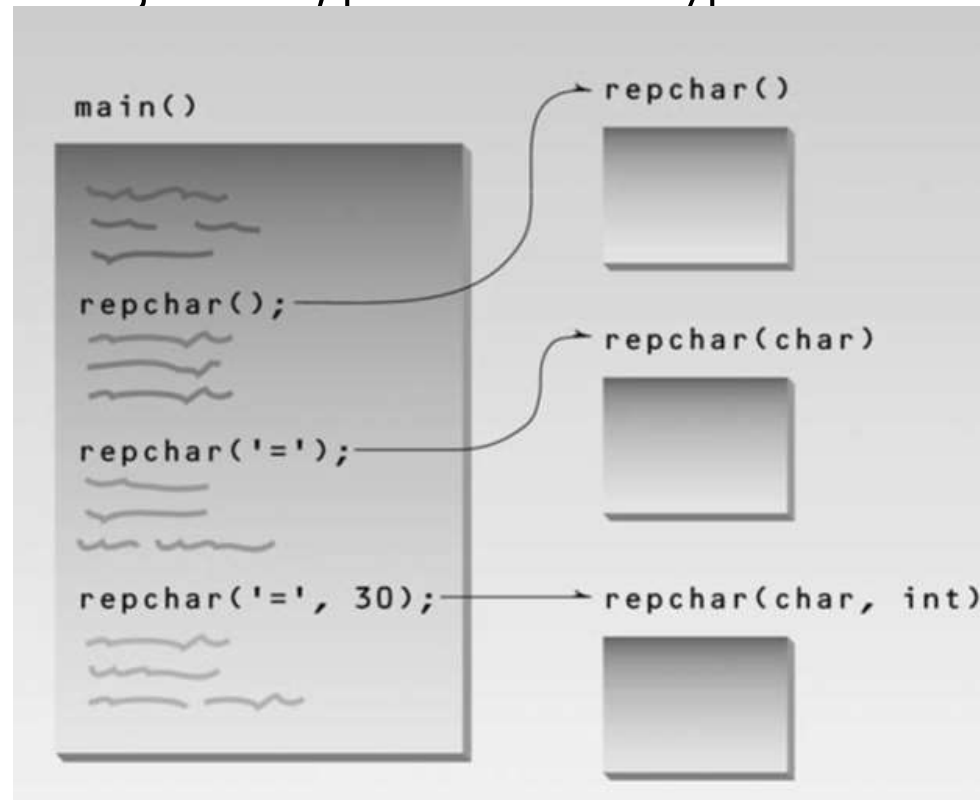
There are three parts required:

the function declaration,
the calls to the function, and
the function definition.

- Q. Write a programme with a function to take the number and the character need to print as user inputs .
- Argument passing
 - By value (float a)
 - By reference (float& a)
 - References don't exist in C, where pointers serve a somewhat similar purpose.
- Return a value
- Q. Modify the above programme by adding another function which take the temperature in Celsius and convert it to Fahrenheit , return the Fahrenheit value.

Overloaded Functions

- An overloaded function appears to perform different activities depending on the kind of data sent to it.
 - with the same name but different numbers of arguments.
 - with the same number of arguments, provided their type is different.



```
// overload.cpp
// demonstrates function overloading
#include <iostream>
using namespace std;
void repchar();           //declarations
void repchar(char);
void repchar(char, int);
int main() {
    repchar();
    repchar('=');
    repchar('+', 30);
    return 0;
}
```

```
void repchar() {
    for(int j=0; j<45; j++)
        cout << '*';
    cout << endl;
}
void repchar(char n) {
    for(int j=0; j<45; j++)
        cout << n;
    cout << endl;
}
void repchar(char n, int m) {
    for(int j=0; j<m; j++)
        cout << n;
    cout << endl;
}
```

Variables

- Scope
- Lifetime
 - Automatic, Static
- Visibility
 - While local variables are defined within functions, global variables are defined outside of any function.

Q.

- Write a function called `power()` that takes a double value for `n` and an int value for `p`, and returns the result(n^p) as a double value. Use a default argument of 2 for `p`, so that if this argument is omitted, the number `n` will be squared.

Thank You !